CSS

LECTURE 06

Box sizing

- By default, the width and height of an element is calculated like this:

  width + padding + border = actual width of an element
  height + padding + border = actual height of an element

```
.div1 {
  width: 300px;
  height: 100px;
  border: 1px solid blue;
}

.div2 {
  width: 300px;
  height: 100px;
  padding: 50px;
  border: 1px solid red;
}
```

This div is smaller (width is 300px and height is 100px).

This div is bigger (width is also 300px and height is 100px).

Box Sizing

- The box-sizing property allows us to include the padding and border in an element's total width and height.
- If you set box-sizing: border-box; on an element, padding and border are included in the width and height.
-

```
.div1 {
  width: 300px;
  height: 100px;
  border: 1px solid blue;
  box-sizing: border-box;
}

.div2 {
  width: 300px;
  height: 100px;
  padding: 50px;
  border: 1px solid red;
  box-sizing: border-box;
}
```

Psedo Class Selectors

- A pseudo-class is used to define a special state of an element.
- For example, it can be used to:
- Style an element when a user mouses over it
- Style visited and unvisited links differently
- Style an element when it gets focus

```
/* unvisited link */
a:link {
  color: #FF0000;
}

/* visited link */
a:visited {
  color: #00FF00;
}

/* mouse over link */
a:hover {
  color: #FF00FF;
}

/* selected link */
a:active {
  color: #0000FF;
}
```

Pseudo-element Selectors

- A CSS pseudo-element is used to style specified parts of an element.
- For example, it can be used to:
- Style the first letter, or line, of an element
- Insert content before, or after, the content of an element
-
```
selector::pseudo-element {
  property: value;
}
```

Pseudo-element Selectors

- The ::first-line pseudo-element is used to add a special style to the first line of a text.
- The ::first-letter pseudo-element is used to add a special style to the first letter of a text.
  - The ::first-line pseudo-element can only be applied to block-level elements.
- The ::before pseudo-element can be used to insert some content before the content of an element.
- The ::after pseudo-element can be used to insert some content after the content of an element.
- The ::marker pseudo-element selects the markers of list items.
- The ::selection pseudo-element matches the portion of an element that is selected by a user.

Attribute Selectors

- The [attribute] selector is used to select elements with a specified attribute.

Selector	Example	Example description
[*attribute*]	[target]	Selects all elements with a target attribute
[*attribute=value*]	[target="_blank"]	Selects all elements with target="_blank"
[*attribute~=value*]	[title~="flower"]	Selects all elements with a title attribute that contains a space-separated list of words, one of which is "flower"
[*attribute|=value*]	[lang|="en"]	Selects all elements with a lang attribute value starting with "en"
[*attribute^=value*]	a[href^="https"]	Selects all <a> elements with a href attribute value starting with "https"
[*attribute$=value*]	a[href$=".pdf"]	Selects all <a> elements with a href attribute value ending with ".pdf"
[*attribute*=value*]	a[href*="w3schools"]	Selects all <a> elements with a href attribute value containing the substring "w3schools"

Styling Forms

- The attribute selectors can be useful for styling forms without class or ID:
-
```
input[type="text"] {
  width: 150px;
  display: block;
  margin-bottom: 10px;
  background-color: yellow;
}

input[type="button"] {
  width: 120px;
  margin-left: 35px;
  display: block;
}
```

Activity

- Open W3schools and try out all examples of attribute selectors.
- https://www.w3schools.com/css/css_attribute_selectors.asp
- Design a website using float to style the layout and navbar elements.

CSS Position

- The position property specifies the type of positioning method used for an element.
- There are five different position values:
  - static
  - relative
  - fixed
  - absolute
  - sticky
- Elements are then positioned using the top, bottom, left, and right properties. However, these properties will not work unless the position property is set first. They also work differently depending on the position value.

CSS Position: static

- Static positioned elements are not affected by the top, bottom, left, and right properties.
- An element with **position: static;** is not positioned in any special way; it is always positioned according to the normal flow of the page

CSS Position: relative

- An element with **position: relative;** is positioned relative to its normal position.
- Setting the top, right, bottom, and left properties of a relatively-positioned element will cause it to be adjusted away from its normal position.
- Other content will not be adjusted to fit into any gap left by the element.

CSS Position: fixed

- An element with **position: fixed;** is positioned relative to the viewport, which means it always stays in the same place even if the page is scrolled.
- The top, right, bottom, and left properties are used to position the element.
- A fixed element does not leave a gap in the page where it would normally have been located.

CSS Position: absolute

- An element with **position: absolute;** is positioned relative to the nearest ***positioned*** ancestor (instead of positioned relative to the viewport, like fixed).
- However; if an absolute positioned element has no positioned ancestors, it uses the document body, and moves along with page scrolling.

CSS Position: sticky

- An element with **position: sticky**; is positioned based on the user's scroll position.
- A sticky element toggles between ***relative*** and ***fixed***, depending on the scroll position.
- It is positioned relative until a given offset position is met in the viewport - then it "sticks" in place (like position:fixed).

```
#sq1
    background-color: red;
    left: 50px;
    top: 50px;
    position: absolute;
}
#sq2 {
    position: relative;
    top: 50px;
    left: 10px;
    background-color: pink;
}
#sq3 {
    background-color: grey;
    position: fixed;
    left: 10px;
    bottom: 50px;
}
#sq4 {
    top: 50px;
    background-color: black;
    position: sticky;
    color: white;
    opacity: 50%;
}
.position {
    position: relative;
    margin-top: 50px;
    height: 500px;
    background-color: bisque;
}
```

MDN Reference

```
<div class="position">
    <h3>CSS Position</h3>
    <h4>The &lt;div&gt; is positioned relative</h4>
    <div class="square" id="sq1">ABSOLUTE</div>
    <div class="square" id="sq2">RELATIVE</div>
    <div class="square" id="sq3">FIXED</div>
    <div class="square" id="sq4">STICKY</div>
</div>
```

Flexbox

- The Flexible Box Layout Module, makes it easier to design flexible responsive layout structure without using float or positioning.
- You need to first define a flex container.
- The flex container becomes flexible by setting the **display** property to **flex.**

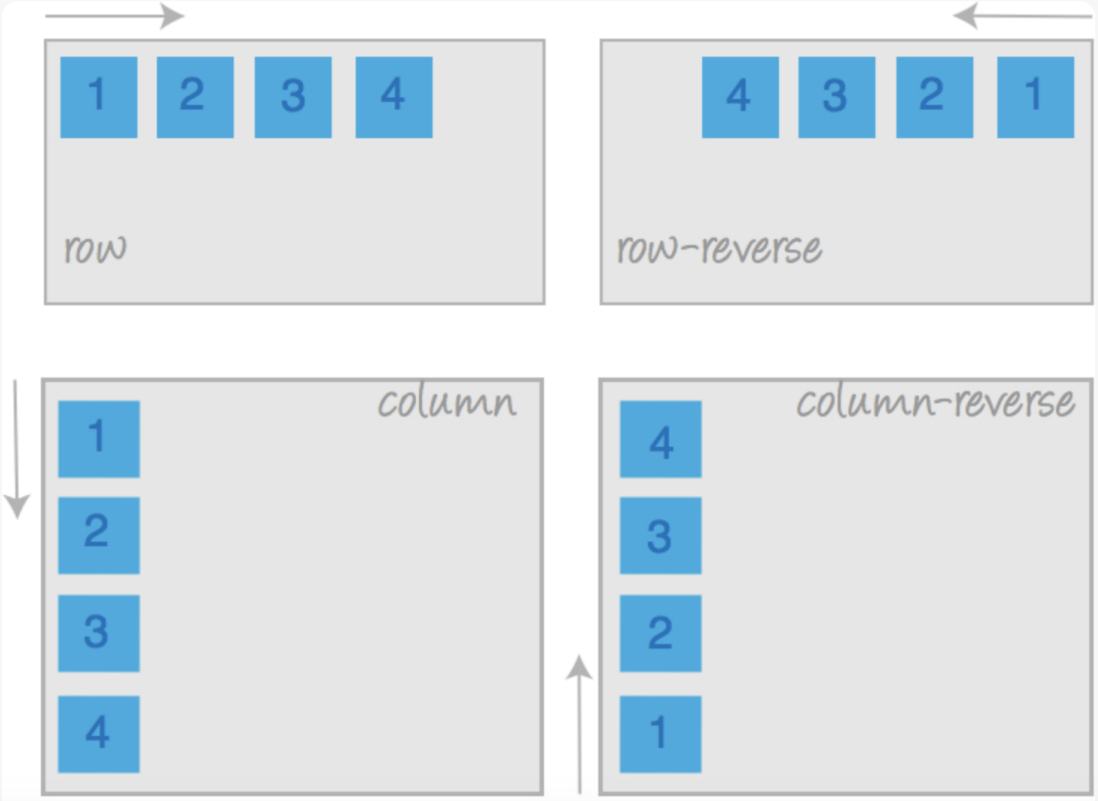
Flexbox

Flexbox

- The flex container properties are:
  - Flex-direction
  - Flex-wrap
  - Flex-flow (shorthand for both flex-direction and flex wrap, e.g flex-flow: row wrap
  - Justify-content
  - Align-items (default value: stretch)
  - Align-content

The flex-direction property

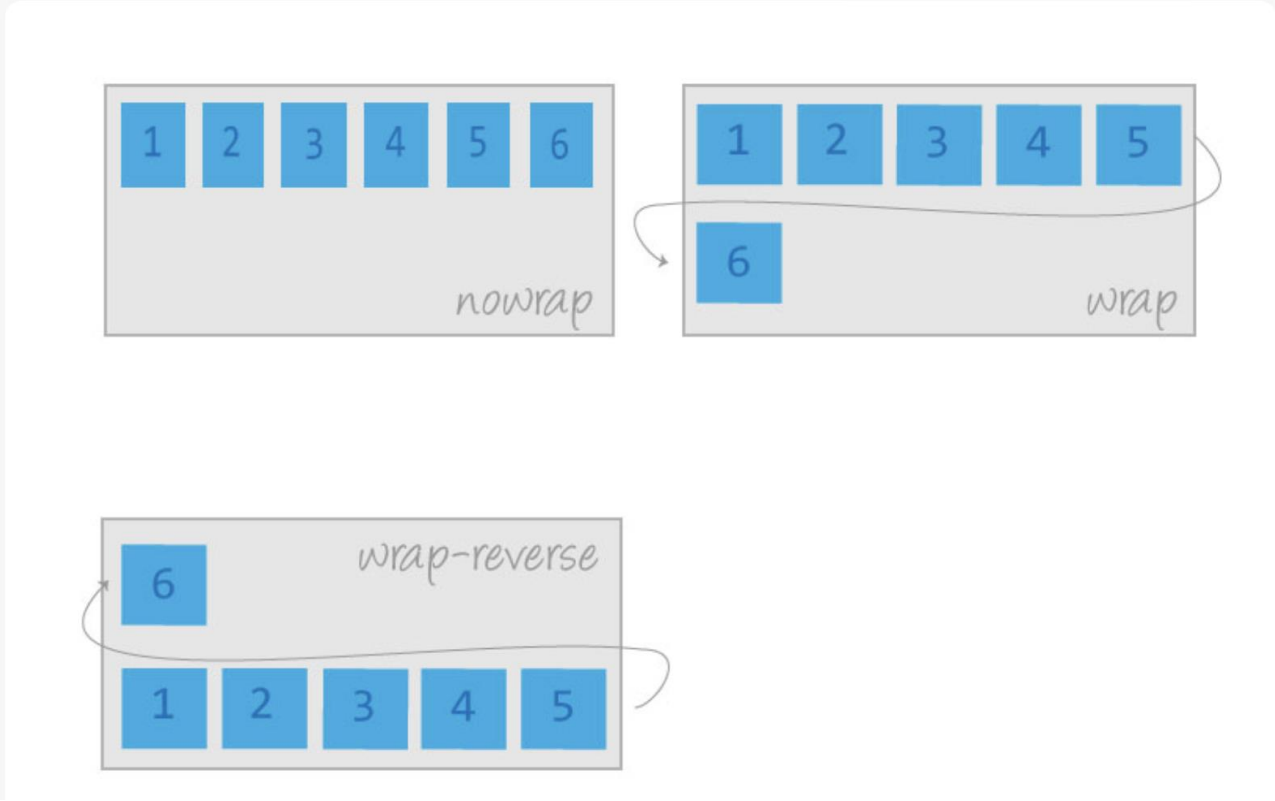
```
flex-direction: row | row-reverse | column | column-reverse
```



The flex-wrap property

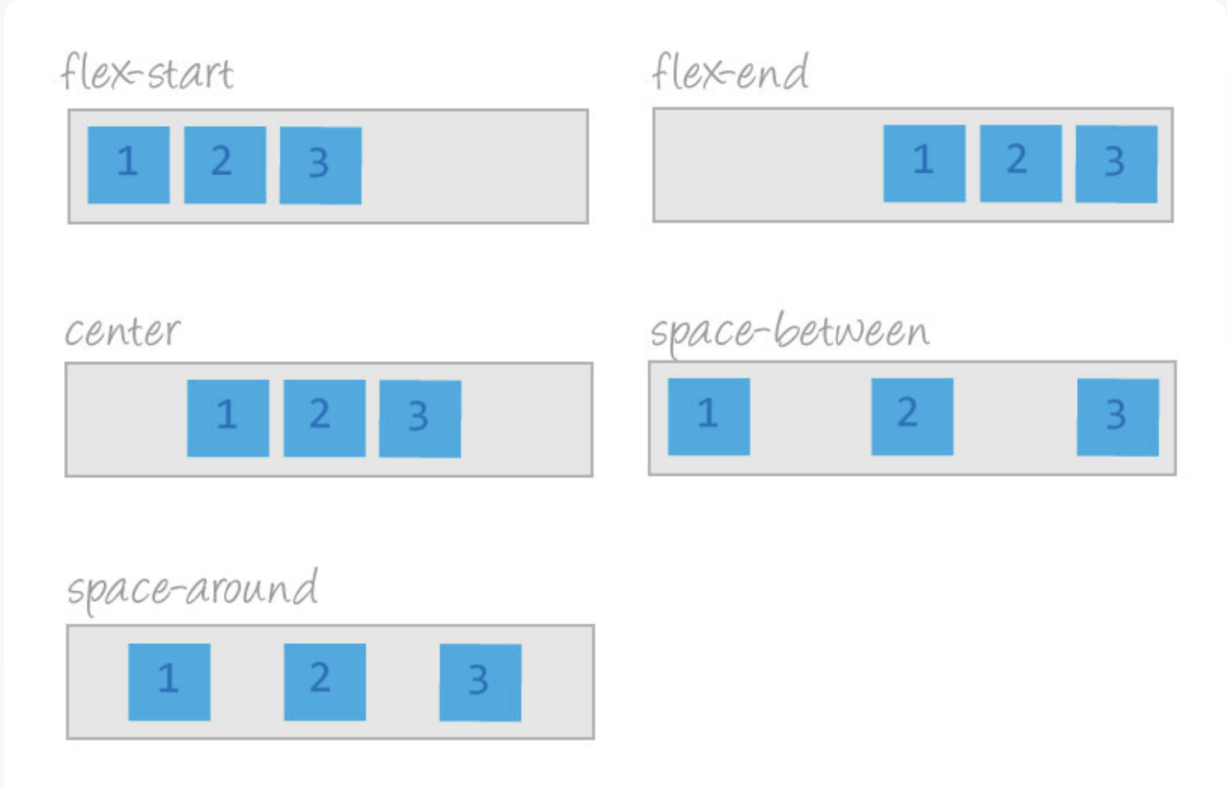
```
flex-wrap: nowrap | wrap | wrap-reverse
```

- The default value of the flex-wrap property is **nowrap**.
- **All flex items stay in a single row** (or column if flex-direction: column is set).
- Items **shrink** if necessary to fit within the container.



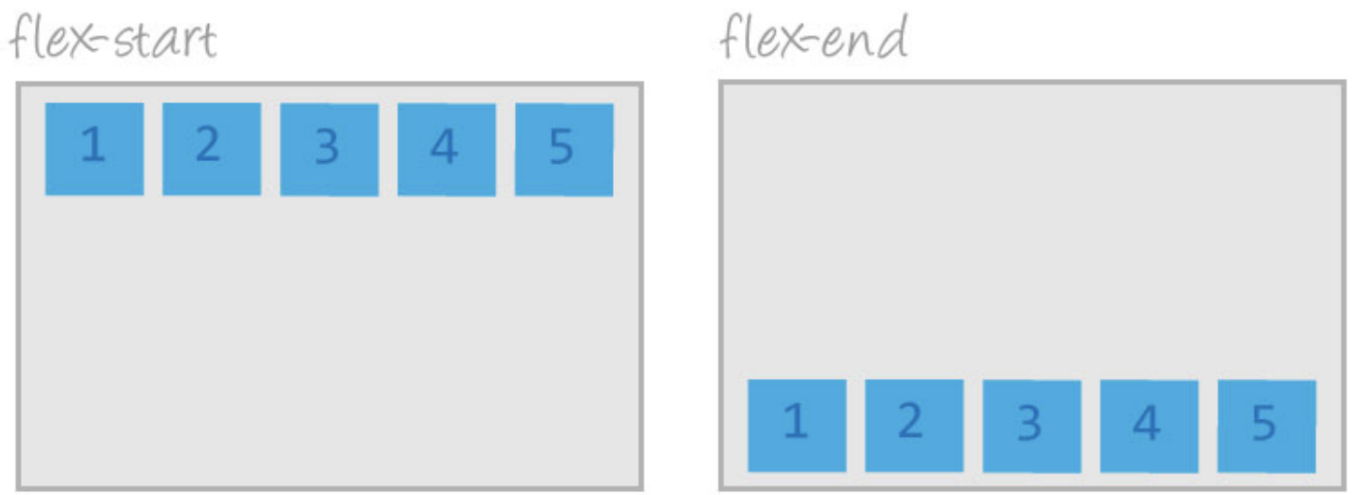
The justify-content property

```
justify-content: flex-start | flex-end | center | space-between | space-around
```

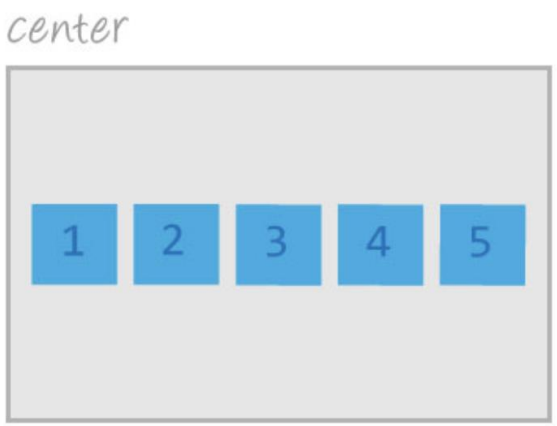


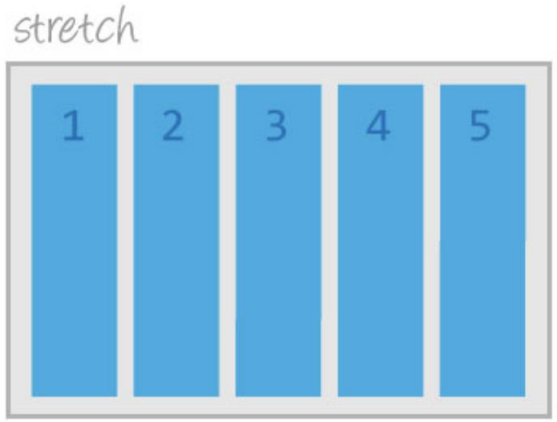
The align-items property

```
align-items: flex-start | flex-end | center | baseline | stretch
```



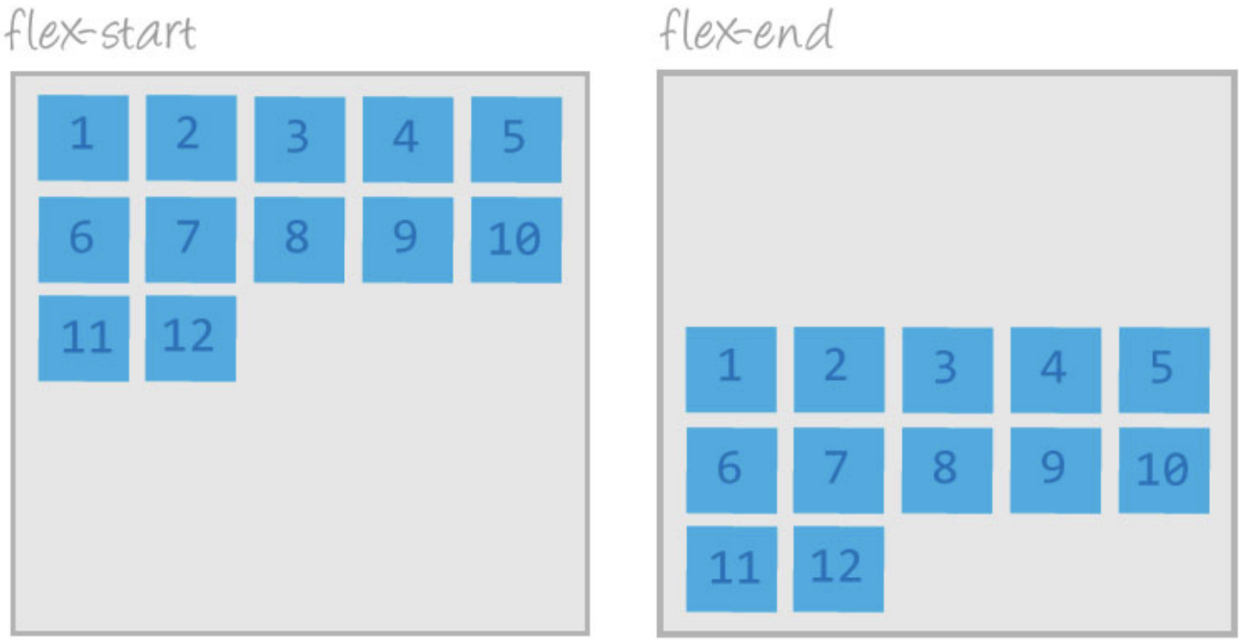
The align-items property



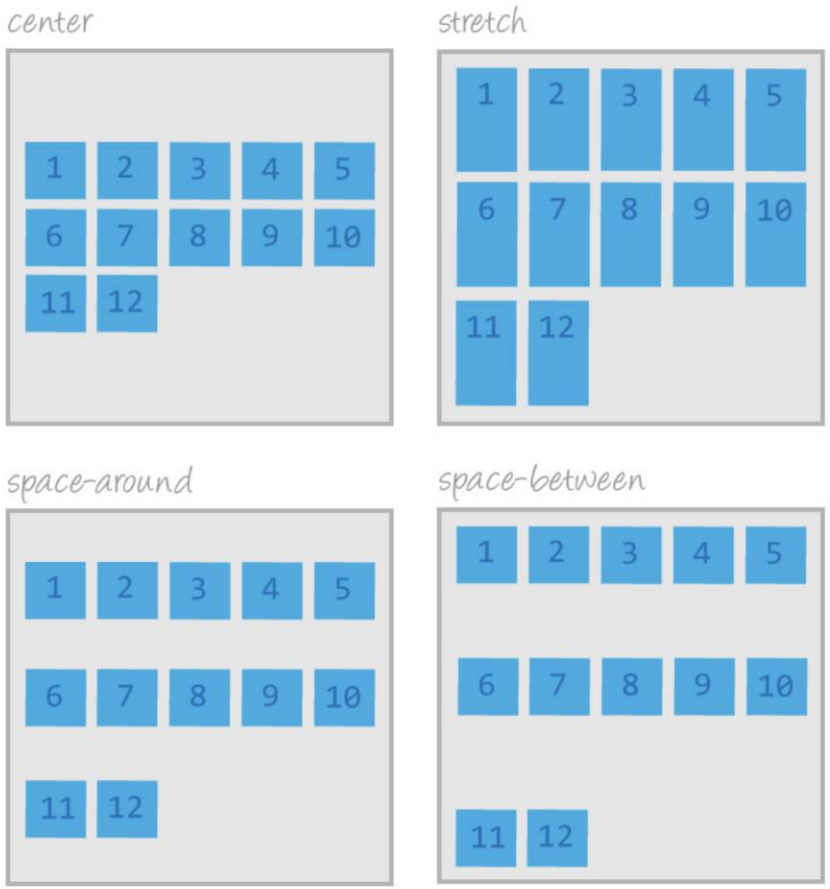


The align-content property

```
align-content: flex-start | flex-end | center | space-between | space-around | stretch
```



The align-content property



Flex Item properties

- Order property
- Align-self property
- Flex-grow
- Flex-shrink

Order property

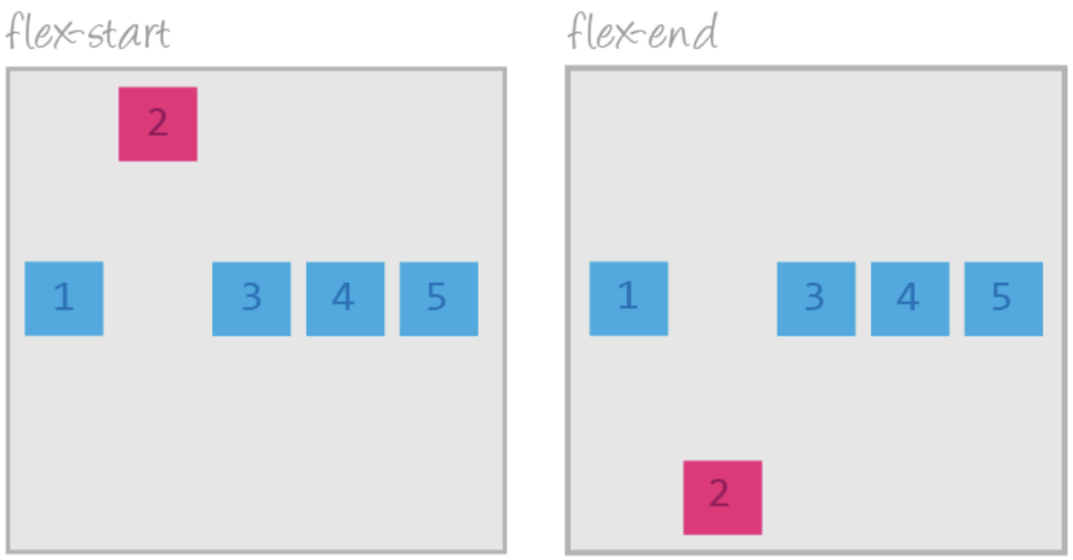
```
.three { order: 7; }
```

```
.three { order: 1; } /* since all items are order: 0; the
item with #3 will be put after them. */
```

Align-self property

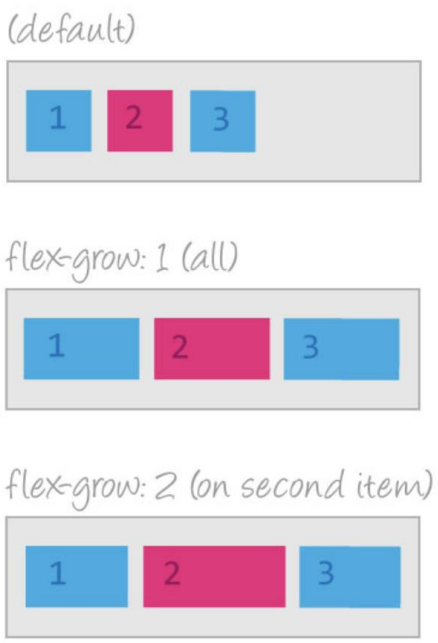
The `align-self` Property

```
align-self: auto | flex-start | flex-end | center | baseline | stretch
```

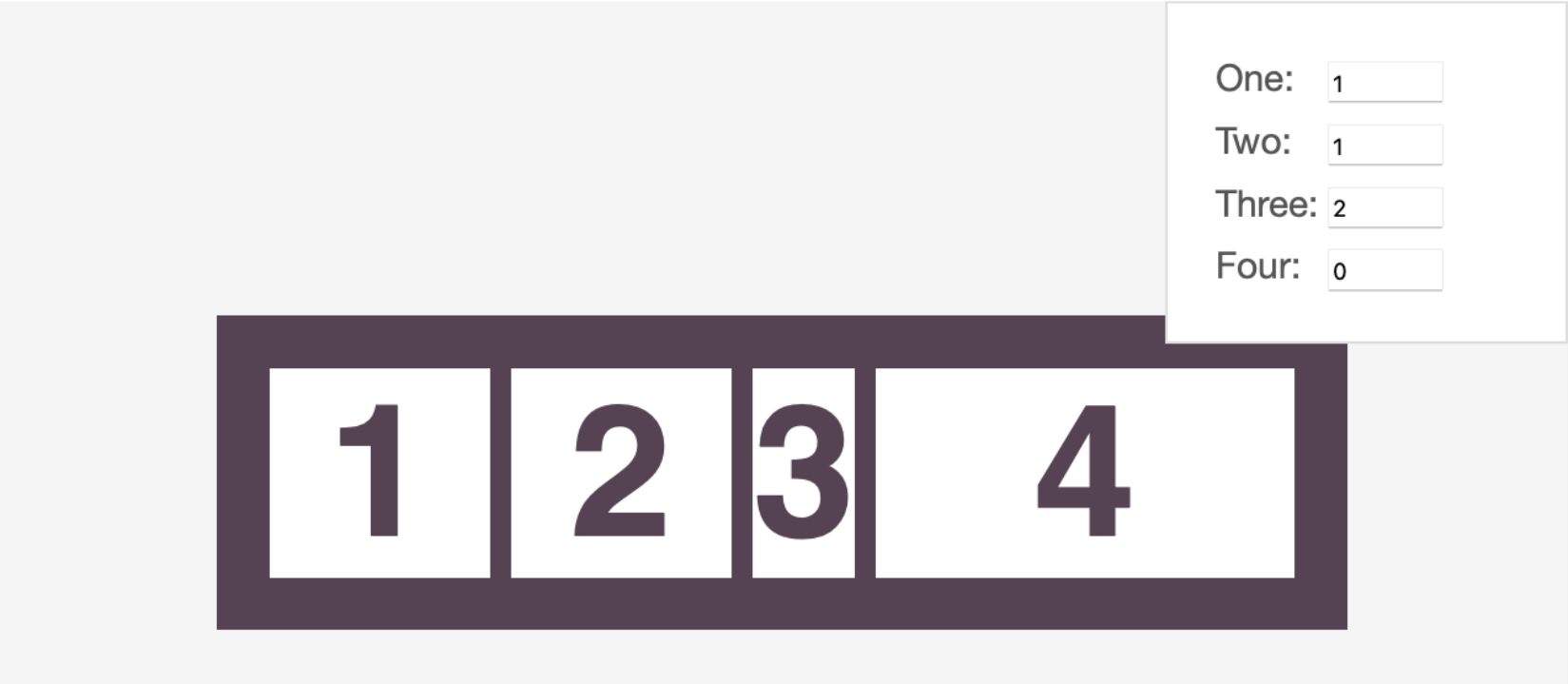


Flex-grow property

```
flex-grow: <number>
```



Flex-shrink property



Activity

- WHAT WE'RE OFFERING

EVENTS ORGANIZED

The Sports Pavilion is a cutting-edge fitness and recreational centre that offers something for everyone. It is a destination that combines relaxation, fitness, and entertainment, making it an all-in-one hub for recreation. The Sports Pavilion offers a vast array of facilities, including a gym, salon and spa, skating rink, volleyball area, basketball court, indoor PC and PS4 gaming, soft play area, snooker area, and a bowling alley. Whether you're looking to unwind, exercise, or have fun, the Sports Pavilion has it all.

Discover More